



YENEPOYA UNIVERSITY

Final Project Report ON CLOUD FORENSICS AUTOMATION FOR E-COMMERCE BREACHES

Student Name: Ardra P V

Industry Mentor: Mr. Shashank Korram

Guided By: Ms. Suzana Sharol

TABLE OF CONTENTS

Section No.	Heading	Page No.
1	Background	4
1.1	Aim	4
1.2	Technologies	4
1.3	Hardware Architecture	5
1.4	Software Architecture	6
2	System	8
2.1	Requirements	9
2.1.1	Functional Requirements	9
2.1.2	User Requirements	9
2.1.3	Environmental Requirements	10
2.2	Design and Architecture	10
2.3	Implementation	12
2.4	Testing	13
2.5	Graphical User Interface (GUI) Layout	14
2.6	Customer Testing	15
2.7	Evaluation	16
3	Snapshots of the Project	17
4	Conclusions	19
5	Further Development or Research	20
6	References	21
7	Appendix	22

EXECUTIVE SUMMARY

The Problem in Cloud Forensics

In modern E-commerce systems, data breaches and unauthorized file modifications are increasingly common. Investigators must analyze large volumes of files to detect tampering or suspicious activity. However, manual analysis of file metadata is time-consuming, inefficient, and prone to errors. This often results in delayed response and increased security risks. In critical situations, such delays can impact business operations and compromise sensitive customer data. Additionally, the absence of automated tools makes it difficult to handle large-scale investigations effectively.

The Automated Forensic Solution

This project introduces the **Cloud Forensics Automation System**, a lightweight platform that automates file metadata analysis. The system extracts important attributes such as creation time, modification time, and file size, and applies detection logic to identify suspicious changes. Files are flagged when modification time exceeds creation time, indicating possible tampering. It also ensures consistent results by applying the same detection logic across all files without human bias.

How the System Works for Users

The system provides a simple workflow where users upload files through a web interface, and the backend automatically processes them. Metadata is extracted, analyzed, and classified into normal or suspicious categories based on predefined logic. The results are displayed in a dashboard with highlighted entries, making it easy to identify issues. Users can also download reports in CSV format for further investigation and documentation.

Key Project Achievements

- Automated forensic analysis, reducing manual effort and investigation time
- Accurate identification of suspicious file modifications using timestamp comparison
- User-friendly dashboard for clear and quick visualization of results
- Secure local processing without dependence on external services
- Easy generation of structured CSV reports for documentation and further analysis
- Simple and efficient system design suitable for real-world usage

1. BACKGROUND

With the rapid growth of cloud-based applications and E-commerce platforms, the risk of data breaches and unauthorized file modifications has increased significantly. Organizations handle large volumes of sensitive data, making them attractive targets for cyberattacks. During such incidents, investigators must quickly identify compromised or tampered files to prevent further damage.

Traditional forensic analysis methods rely on manual inspection of file metadata, which is time-consuming, error-prone, and requires specialized expertise. These limitations make it difficult to respond quickly to security incidents. Therefore, there is a need for an automated system that can simplify and accelerate the forensic analysis process. This project addresses this need by providing an efficient and user-friendly solution.

1.1 Aim

The main aim of this project is to design and develop a system that automates the process of file metadata analysis for forensic investigations. The system focuses on detecting suspicious file modifications by analyzing timestamps and identifying abnormal patterns.

It also aims to provide quick and accurate forensic insights through an easy-to-use interface. Additionally, the system generates structured reports that help investigators document findings and perform further analysis. Overall, the project aims to improve efficiency, reduce manual effort, and support faster incident response.

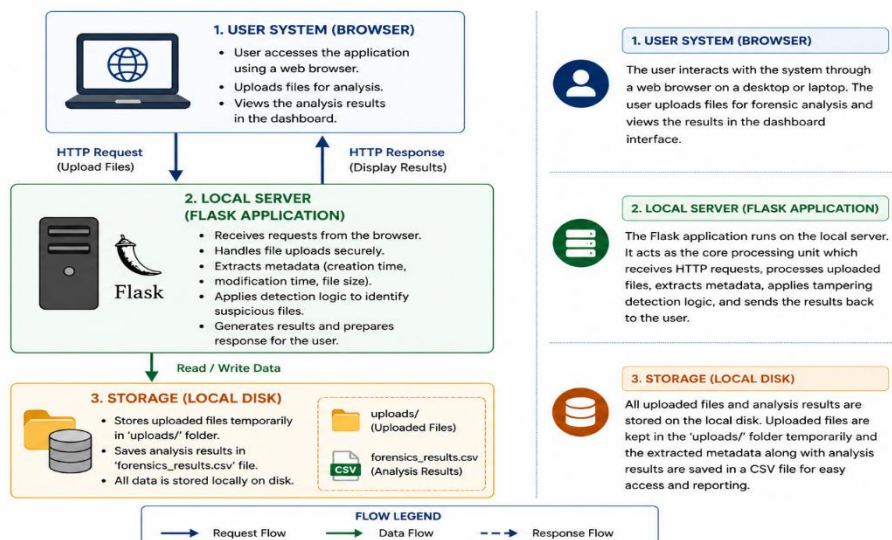
1.2 Technologies

➤ Frontend

- **HTML5:** Used to create the structure of web pages such as the upload form and dashboard layout.
- **CSS3:** Provides styling, layout design, and responsiveness to make the interface visually appealing.
- **JavaScript:** Adds interactivity, such as handling file uploads and improving user experience.

- **Jinja2:** A templating engine used with Flask to dynamically display data (like analysis results) in HTML pages.
- Backend
- **Python 3:** Core programming language used to implement the application logic and data processing.
- **Flask:** A lightweight web framework used to handle routing, file uploads, and communication between frontend and backend.
- **Werkzeug:** Provides utilities for secure file handling, including safe filename processing to prevent security risks.
- Storage
- **CSV File (forensics_results.csv):** Stores analyzed file metadata in a structured format for easy reading and reporting.
- **Local File System:** Used to temporarily store uploaded files before processing and analysis.

1.3 Hardware Architecture



The hardware architecture follows a **client-server model** designed for local execution. It consists of the user system, local server, and storage, all working together to process and analyze files efficiently.

The **User System (Browser)** acts as the interface through which users interact with the application. It allows users to upload files and view the analysis results using a web browser. The browser sends HTTP requests to the server and displays the processed results returned by the system.

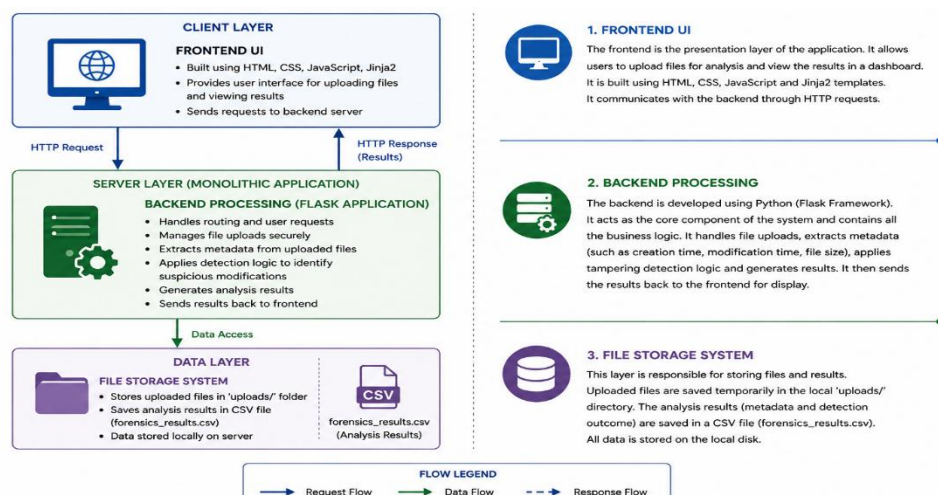
The **Local Server (Flask Application)** performs the core processing tasks. It receives requests from the browser, handles file uploads securely, extracts metadata, and applies tampering detection logic to analyze the files. This server acts as the central processing unit of the system.

The **Storage (Local Disk)** is used to store uploaded files temporarily and save the analysis results in a CSV file. This ensures that all data is processed and stored locally, making the system simple and secure.

Additionally, the system ensures smooth data flow between components, where user input is processed by the server and results are returned in real time. Since all components operate within a local environment, the system provides faster performance and better data privacy compared to cloud-based solutions.

The architecture is designed to be simple and efficient, making it suitable for standalone usage. However, it can be extended in the future by integrating cloud storage or distributed systems for handling larger datasets and multiple users.

1.4 Software Architecture



The software architecture follows a **client-server model** using a **monolithic design**, where all components are combined into a single application. This makes the system simple to develop, run, and maintain, especially for small-scale projects.

The **Frontend UI** is responsible for user interaction. It allows users to upload files and view analysis results through a web interface. It sends requests to the backend and displays the results received from the server.

The **Backend Processing layer** handles the core functionality of the system. It manages file uploads, extracts metadata such as creation and modification time, and applies detection logic to identify suspicious files. After processing, it sends the results back to the frontend.

The **File Storage System** stores uploaded files temporarily and saves the analysis results in a CSV file. This approach keeps the system lightweight and avoids the need for a database, while still allowing easy access to results.

The system ensures smooth communication between components using HTTP requests and responses. Each layer performs a specific role, allowing better organization and maintainability of the application.

Although the system uses a monolithic structure, it provides good performance for small-scale usage. In the future, it can be enhanced by separating components into independent services and integrating a database for improved scalability and efficiency.

2. SYSTEM

The system is divided into several functional modules, each responsible for a specific task in the forensic analysis process:

1. File Upload Module: Handles user interaction for uploading files through the web interface. It supports multiple file uploads and ensures secure handling of files using validation and sanitization techniques.

2. Metadata Extraction Module: Processes uploaded files and extracts important metadata such as creation time, modification time, and file size using system-level functions.

3. Detection Engine: Analyzes the extracted metadata and applies tampering detection logic. Files are flagged as suspicious if the modification time is significantly later than the creation time.

4. Dashboard Module: Displays the analysis results in a structured tabular format. Suspicious files are highlighted to help users quickly identify potential issues.

5. Report Generation Module: Stores the analysis results in a CSV file and allows users to download the report for further investigation and documentation.

The system operates through a structured workflow where each module performs a specific function in sequence. Once the user uploads files, they are temporarily stored and passed to the metadata extraction module, where key attributes such as timestamps and file size are collected. This data is then analyzed by the detection engine to identify suspicious patterns based on predefined logic.

After processing, the results are displayed in the dashboard module in a clear and organized format, with suspicious files highlighted for easy identification. The system also stores the analysis results in a CSV file, which can be downloaded for further investigation and documentation. Overall, this workflow ensures efficient processing, reduces manual effort, and provides quick insights into file activity.

2.1 Requirements

2.1.1 Functional Requirements

- **Upload Files:**
The system must allow users to upload one or more files through a web interface for analysis. It should support multiple file formats and ensure secure handling during upload.
- **Extract Metadata:**
The system must automatically extract file metadata such as creation time, modification time, and file size using system-level functions.
- **Detect Suspicious Files:**
The system must analyze the extracted metadata and identify suspicious files based on predefined logic, such as abnormal differences between creation and modification timestamps.
- **Display Results:**
The system must present the analysis results in a clear and structured dashboard, highlighting suspicious files for easy identification.
- **Download CSV Report:**
The system must allow users to download the analysis results in CSV format for further investigation, documentation, or sharing.

2.1.2 User Requirements

- **Simple UI:**
The system should provide an easy-to-use and intuitive interface so that users can upload files and view results without requiring technical expertise.
- **Fast Response:**
The system should process uploaded files quickly and display results without significant delay, ensuring efficient analysis and better user experience.

- **Clear Results:**

The system should present analysis results in a clear and understandable format, with suspicious files highlighted for easy identification.

2.1.3 Environmental Requirements

- **Python Installed:**

The system requires Python to be installed on the local machine to run the Flask backend and execute the application.

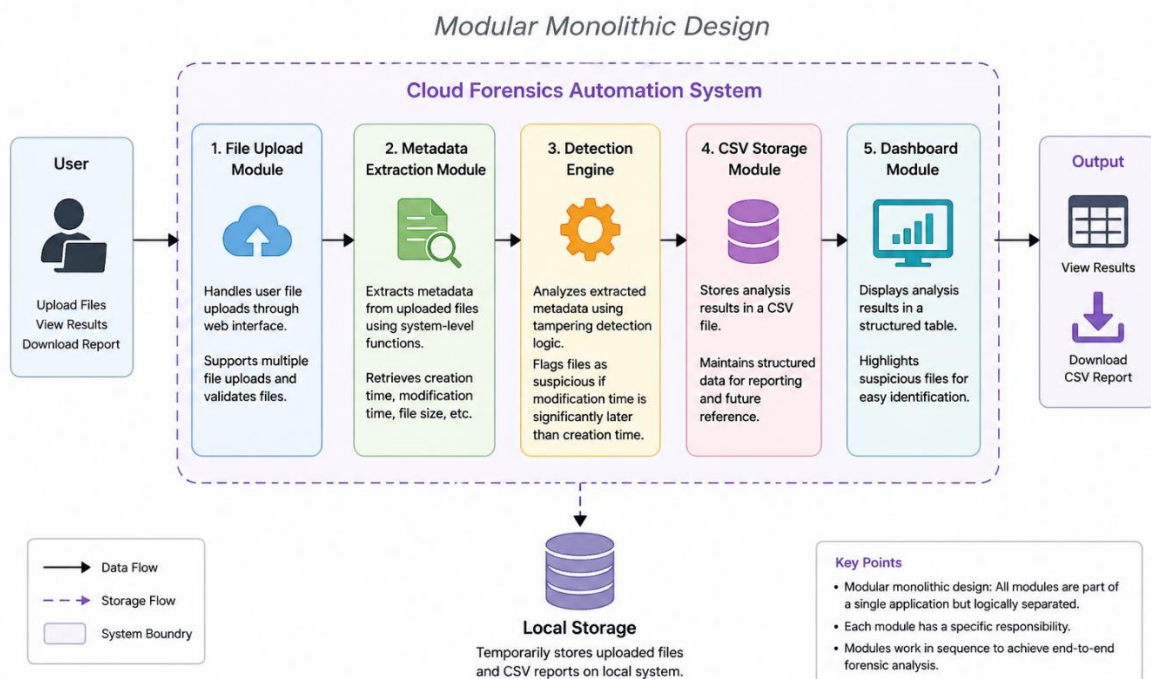
- **Browser Access:**

A web browser is required to access the user interface, upload files, and view the analysis results.

- **Local System Storage:**

The system requires sufficient local storage to temporarily store uploaded files and save the analysis results in a CSV file.

2.2 Design and Architecture



The system follows a **modular monolithic design**, where all components are part of a single application but are logically divided into separate modules. This approach keeps the system simple to develop and maintain while still allowing clear separation of responsibilities.

In this design, each module performs a specific task but works together within the same system. Since all modules are integrated into one application, communication between them is fast and efficient, without the need for external services or complex configurations.

The system is organized into the following modules:

The **File Upload Module** is responsible for handling user input. It allows users to upload files through the web interface and ensures that files are securely received and stored for processing.

The **Metadata Extraction Module** processes the uploaded files and extracts important information such as creation time, modification time, and file size. This data is essential for further analysis.

The **Detection Engine** analyzes the extracted metadata and applies predefined logic to identify suspicious files. It compares timestamps and flags files that show abnormal modification patterns.

The **CSV Storage Module** stores the analysis results in a structured CSV file. This helps in maintaining records and allows users to download the data for further use.

The **Dashboard Module** presents the final results in a clear and user-friendly format. It displays all analyzed files and highlights suspicious ones, making it easy for users to understand the output.

2.3 Implementation

The implementation of the system is carried out using **Python and the Flask framework**, following a structured sequence of steps to ensure proper functionality and smooth data processing. Flask acts as the backend server, handling all user interactions and coordinating the overall workflow of the system.

The process begins with setting up the Flask application and configuring essential parameters such as the upload folder and file size limits. Routes are defined to manage different operations, including file upload, result display, and report download. These routes enable communication between the frontend interface and the backend processing system.

A dedicated upload route is created to receive files from the user through the web interface. When files are uploaded, the system validates them and securely saves them using filename sanitization techniques to prevent security risks such as unauthorized file access or path traversal.

Once the files are stored, the system performs metadata extraction using Python's `os` module. It retrieves important attributes such as creation time, modification time, and file size for each uploaded file. These values are then formatted into a readable structure for further analysis.

The extracted data is processed using detection logic, where the system compares timestamps to identify suspicious files. If the modification time is significantly later than the creation time, the file is flagged as suspicious. This step helps in detecting potential tampering or unauthorized changes.

After analysis, the results are stored in a CSV file, ensuring that the data is organized and can be easily accessed or downloaded. Each record contains file details along with its status, making the output structured and easy to interpret.

Finally, the system renders a dashboard view using templates, where users can see the results in a clear and structured format. Suspicious files are highlighted to allow quick identification. This completes the workflow from file upload to result visualization, providing a simple and efficient forensic analysis system.

2.4 Testing

Testing was conducted to ensure the system functions correctly, securely, and efficiently. The testing process covered different levels of the application to validate each component and overall system performance.

Unit Testing

Unit testing focused on verifying individual components of the system independently. Each module, such as file upload, metadata extraction, and detection logic, was tested in isolation.

For example, the metadata extraction module was tested to ensure it correctly retrieves creation time, modification time, and file size. The detection logic was also tested with different timestamp values to confirm that suspicious files are correctly identified based on predefined conditions.

Integration Testing

Integration testing ensured that all modules work together properly. The interaction between file upload, metadata extraction, detection engine, and dashboard display was tested as a complete workflow.

Test cases verified that when files are uploaded, they are processed correctly, metadata is extracted, results are stored in the CSV file, and displayed accurately in the dashboard without errors or delays.

Security Testing

Security testing was performed to ensure safe handling of user inputs and file operations. The system was tested against common vulnerabilities:

- **File Upload Validation:** Tested uploading invalid or unsupported file types to ensure the system rejects unsafe inputs.
- **Path Traversal Protection:** Verified that filename sanitization prevents access to restricted directories.
- **Input Handling:** Ensured user inputs are safely processed without causing system crashes or unexpected behaviour.

Performance Testing and User Experience

The system was tested for performance and usability under normal conditions. Multiple files were uploaded to check system response time and processing speed.

The dashboard was evaluated for clarity and responsiveness, ensuring that results are displayed quickly and in an understandable format. The system provided smooth interaction without noticeable delays, confirming its efficiency for small-scale usage.

2.5 Graphical User Interface (GUI) Layout

The graphical user interface (GUI) of the system is designed to be simple, intuitive, and easy to use, ensuring that users can perform forensic analysis without requiring technical expertise. The interface is divided into two main pages: the Home Page and the Dashboard Page, each serving a specific purpose in the workflow.

The **Home Page** acts as the starting point of the system. It provides a clean layout with a drag-and-drop upload area as well as a file selection option. Users can upload one or multiple files easily, and the interface provides clear buttons to start the analysis. The design focuses on simplicity so that users can quickly understand how to interact with the system without confusion.

Once the files are processed, the system redirects users to the **Dashboard Page**, where all analysis results are displayed. The results are presented in a structured table format, including details such as file name, creation time, modification time, file size, and status. This organized layout helps users quickly scan and understand the data.

A key feature of the dashboard is the **visual highlighting of suspicious files**, which allows users to immediately identify potential issues. Instead of manually checking each file, users can focus directly on flagged entries, improving efficiency and reducing effort.

The GUI also supports smooth interaction and quick response, ensuring that results are displayed without delay. The overall design maintains consistency in layout, colors, and structure, providing a professional and user-friendly experience

2.6 Customer Testing

Customer testing was conducted to evaluate the usability, performance, and reliability of the system in a real usage scenario. A group of users interacted with the application by uploading different files, analyzing results, and navigating through the interface to complete typical tasks.

The testing focused on key aspects such as **ease of use, system speed, and accuracy of results**. Users were able to understand the interface quickly due to its simple design and clear workflow. The upload process and navigation between pages were smooth, requiring minimal instructions, which indicates good usability.

In terms of performance, the system demonstrated **fast processing speed**, even when multiple files were uploaded. The time taken for metadata extraction and result generation was minimal, providing a responsive experience to users. This ensured that users did not experience delays during analysis.

The accuracy of the system was also evaluated by testing files with different timestamp conditions. The system correctly identified suspicious files based on the implemented detection logic, and the results were consistent across multiple test cases. This shows that the detection mechanism is reliable for basic forensic analysis.

User feedback was overall positive. Users reported that the system is **easy to use**, with a clean and intuitive interface. The **visual highlighting of suspicious files** was particularly useful, as it allowed quick identification of important results without manual inspection. The ability to download reports also added value for further analysis.

2.7 Evaluation

The system was evaluated based on its performance, usability, and overall effectiveness in detecting suspicious file modifications. The evaluation highlights both the strengths and limitations of the system.

Advantages

- The system provides **fast analysis**, as it processes files quickly and generates results without noticeable delay. This improves efficiency and reduces the time required for forensic investigation.
- The design of the system is **simple and user-friendly**, making it easy to use even for non-technical users. The clear interface and structured workflow help users perform tasks without confusion.
- Another major advantage is that the system has **no external dependencies**, as it operates entirely on a local environment. This ensures better data privacy and eliminates the need for internet connectivity or third-party services.

Disadvantages

- The system is designed as a **single-user application**, which limits its usage in multi-user or enterprise environments. It does not support concurrent access or shared analysis.
- There is also **limited scalability**, as the system relies on local storage and a CSV file instead of a database. This makes it less suitable for handling large datasets or high workloads.
- Additionally, the system has **OS dependency for timestamps**, since file creation time behaves differently across operating systems. This may affect the accuracy of detection results in certain environments.

3. SNAPSHOTS OF THE PROJECT

1.Upload Page

Cloud Forensics

UploadDashboard

Cloud Forensics Automation

Simulates collecting evidence of file tampering in an E-commerce data breach scenario.
Upload files to record their **Creation** and **Last Modified** dates into a secure CSV report.

Select Files for Analysis

Click to browse or drag & drop
Supports any file type • Up to 50 MB per upload

Upload & Analyse

Product Files
Detects if e-commerce product data files were secretly modified

Payment Scripts
Flags payment script changes that may indicate a breach

Malicious Uploads
Identifies suspiciously modified or backdated uploaded files

2. Dashboard Page

Cloud Forensics

UploadDashboard

✔ Successfully analysed 3 files. Results saved.

Evidence Log

New Scan

Download Report

Clear Records

3
TOTAL FILES

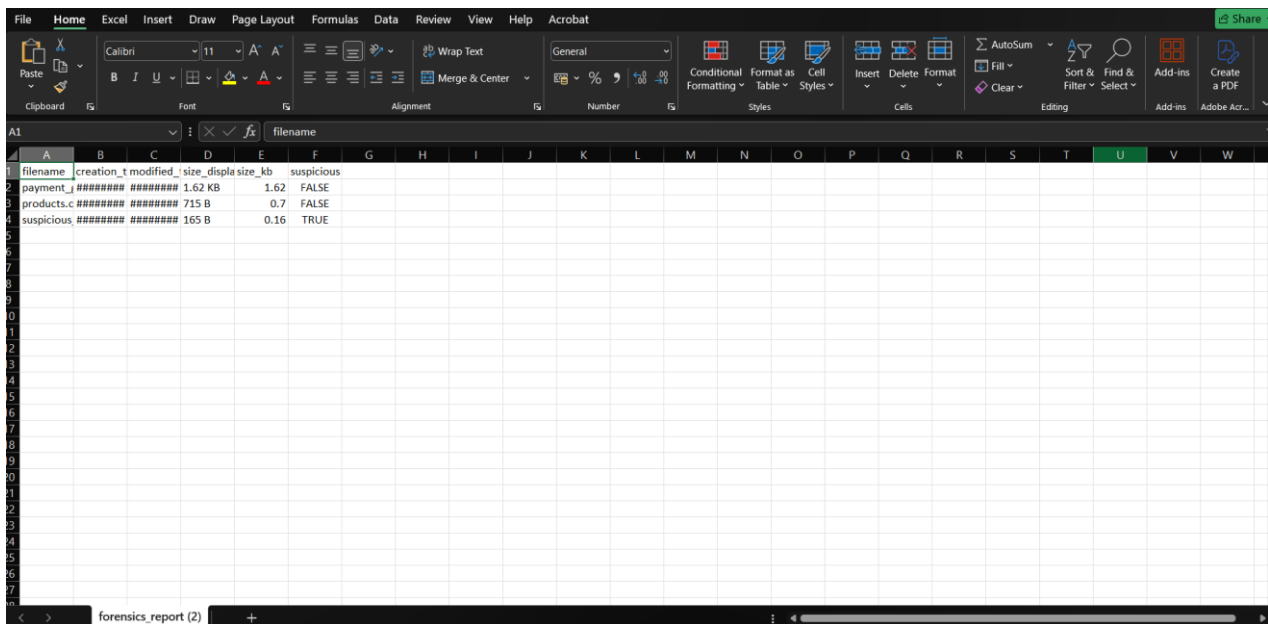
1
SUSPICIOUS

2
CLEAN

#	File Name	Creation Time	Last Modified	Size	Status
1	payment_gateway.js	2026-04-28 21:17:14	2026-04-28 21:17:14	1.62 KB	✔ Normal
2	products.csv	2026-04-28 21:17:14	2026-04-28 21:17:14	715 B	✔ Normal
3	suspicious_test.txt	2026-04-28 21:17:14	2026-04-28 21:17:14	165 B	⚠ Suspicious

Evidence auto-logged to: forensics_results.csv

3. CSV Report



filename	creation_t	modified	size_displa	size_kb	suspicious
payment_1	#####	#####	1.62 KB	1.62	FALSE
products.c	#####	#####	715 B	0.7	FALSE
suspicious	#####	#####	165 B	0.16	TRUE

4. CONCLUSIONS

The project successfully demonstrates how forensic analysis can be automated using simple and efficient tools. The **Cloud Forensics Automation System** provides a practical solution for identifying suspicious file modifications by analyzing file metadata. By automating tasks such as metadata extraction and tampering detection, the system reduces the need for manual inspection and improves the speed of forensic investigations.

The system allows users to upload files, process them through a structured workflow, and view results in a clear and organized dashboard. Suspicious files are highlighted, making it easy for users to identify potential issues without analyzing each file individually. The ability to generate and download reports in CSV format further enhances the usefulness of the system for documentation and analysis.

One of the key strengths of the project is its simplicity and ease of use. The system is designed with a user-friendly interface, making it accessible even to users with limited technical knowledge. Additionally, the use of local processing ensures data privacy and eliminates the need for external dependencies, making the system reliable and secure for standalone usage.

However, the system also has certain limitations. It is designed as a single-user application and may not be suitable for large-scale or multi-user environments. The reliance on local storage and CSV files limits scalability, and the detection logic depends on operating system behaviour for timestamps, which may affect consistency across platforms.

Despite these limitations, the project provides a strong foundation for further development. Future enhancements could include integrating a database for better data management, adding user authentication for multi-user support, and implementing advanced detection techniques for improved accuracy.

Overall, the project highlights the importance of automation in digital forensics and demonstrates how simple technologies can be used to build effective and practical solutions for real-world problems.

5. FURTHER DEVELOPMENT OR RESEARCH

- The current system can be further improved by adding new features and technologies to make it more powerful and scalable.
- **Authentication** can be added to allow users to create accounts and log in securely. This will help protect access to the system and ensure that only authorized users can upload and view data.
- Using a **database such as PostgreSQL** instead of a CSV file will improve data management. It will allow the system to handle large amounts of data efficiently and support better querying, storage, and retrieval of results.
- The system can also be enhanced through **cloud deployment**, where it can be hosted on platforms like AWS or Azure. This will make the application accessible from anywhere and allow multiple users to use it at the same time.
- Introducing **AI-based anomaly detection** can improve the accuracy of identifying suspicious files. Instead of relying only on simple timestamp comparison, the system can learn patterns and detect more complex or hidden threats.
- Finally, adding **multi-user support** will allow multiple users to access and use the system simultaneously. Each user can have separate data and reports, making the system suitable for real-world organizational use.

6. REFERENCES

1. Flask Documentation
<https://flask.palletsprojects.com/>
2. Python OS Module Documentation
<https://docs.python.org/3/library/os.html>
3. Python CSV Module Documentation
<https://docs.python.org/3/library/csv.html>

7. APPENDIX

The appendix provides additional supporting materials related to the project, which help in better understanding the implementation and results. It includes sample data, important code snippets, and the overall project structure.

The **sample CSV file** represents the output generated by the system after analyzing uploaded files. It contains details such as file name, creation time, modification time, file size, and suspicious status. This file helps demonstrate how the system stores and presents analysis results in a structured format. It also serves as proof of the system's functionality and can be used for further analysis or documentation.

The appendix also includes **code snippets** that highlight key parts of the implementation. These may include sections such as file upload handling, metadata extraction using the os module, and the detection logic used to identify suspicious files. These snippets provide insight into how the system works internally without including the entire source code, making it easier to understand the core functionality.

Additionally, the **project structure** is provided to give an overview of how the files and folders are organized within the system. This typically includes directories such as the main application file, templates folder for frontend pages, static files for styling, uploads folder for storing files, and the CSV file for storing results. A clear project structure helps in understanding how different components of the system are connected and managed.

Overall, the appendix supports the main report by providing practical examples and technical details. It enhances clarity, improves understanding of the system, and acts as a reference for anyone reviewing or using the project.